

Kishore Narayanan
JHED ID: knaraya8@jh.edu

Module 5 Report
Software Assurance and Secure SQL (SQLi Defense)

Project Overview

This project expands upon the Grad Cafe Analytics application by focusing on software assurance, secure database access, dependency analysis, vulnerability scanning, reproducible environments, and continuous integration.

The objective of Module 5 was to improve the security and maintainability of the application by implementing SQL injection defenses, reducing database privileges, externalizing configuration through environment variables, analyzing project dependencies, performing vulnerability scanning, and automating quality checks through GitHub Actions.

The completed solution includes:

- SQL injection protection using parameterized queries and psycopg SQL composition
- Query limit validation and enforcement
- Dedicated database application user with restricted permissions
- Environment-based database configuration
- Dependency graph generation using pydeps
- Packaging support using setuptools
- Vulnerability scanning using Snyk
- Automated CI validation using GitHub Actions
- 100% test coverage with Pytest
- 10.00/10 pylint score

Step 1 – Pylint

Pylint was used to identify code quality issues and improve maintainability. Existing warnings were resolved through code cleanup, import organization, naming improvements, and removal of unused variables and imports.

The project was configured to require a perfect pylint score using:

```
pylint src --fail-under=10
```

Final Result:

- Pylint Score: 10.00 / 10.00

```
• (.venv) kishore.narayanan@Kishores-MacBook-Air module_5 % pylint src --fail-under=10
-----
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

This ensures that all submitted source code meets a consistent quality standard before deployment or release.

Step 2 – SQL Injection Defenses

The original application contained SQL statements that could potentially become vulnerable if table names, column names, or query limits were supplied dynamically without validation.

To improve security, SQL execution was refactored to use psycopg's safe SQL composition utilities. Dynamic identifiers are now handled using `sql.Identifier()` and SQL statements are constructed using `sql.SQL()`. This prevents malicious input from being interpreted as executable SQL.

In addition, all user-supplied values are passed using parameterized queries rather than string concatenation. Parameterized queries ensure that values are treated as data rather than executable SQL commands.

Example security improvements included:

- Use of `sql.SQL()` for query construction
- Use of `sql.Identifier()` for table and column names
- Parameterized query execution
- Elimination of direct SQL string concatenation
- Validation of dynamic query inputs

Query Limit:

A limit validation function was implemented to prevent excessive query execution and reduce the risk of abuse.

The application now enforces:

- Minimum query limit: 1
- Maximum query limit: 100
- Default limit when invalid input is supplied

Invalid, negative, excessively large, or malformed values are automatically replaced with safe values before query execution.

This additional validation layer helps protect the database while ensuring predictable application behavior.

Step 3 – Database Hardening

Database configuration was refactored to remove hard-coded settings and move sensitive information into environment variables.

A .env.example file was created to document required configuration values while preventing actual credentials from being stored in source control.

Example: DATABASE_URL=postgresql://username:password@localhost/gradcafe

The .gitignore file was updated to exclude:

```
.env  
.venv/  
.pytest_cache/  
.coverage  
__pycache__/
```

This prevents secrets, temporary files, virtual environments, and local development artifacts from being committed to the repository.

Least-Privilege Database User:

A dedicated application database account (gradcafe_app) was created.

The following permissions were granted:

```
GRANT CONNECT ON DATABASE gradcafe TO gradcafe_app;  
GRANT USAGE ON SCHEMA public TO gradcafe_app;  
GRANT SELECT, INSERT, UPDATE ON TABLE applicants TO gradcafe_app;
```

```
(.venv) kishore.narayanan@Kishores-MacBook-Air module_5 % psql -d gradcafe -c "\du gradcafe_app"  
List of roles  
Role name | Attributes  
-----+-----  
gradcafe_app |
```

Least-privilege database user configuration

```
(.venv) kishore.narayanan@Kishores-MacBook-Air module_5 % psql -d gradcafe -c "\dp applicants"  
Access privileges  
Schema | Name | Type | Access privileges | Column privileges | Policies  
-----+-----+-----+-----+-----+-----  
public | applicants | table | "kishore.narayanan"=arwdDxtm/"kishore.narayanan"+  
gradcafe_app=arw/"kishore.narayanan" | |  
(1 row)
```

Database permissions granted to application account

The following permissions were intentionally not granted:

- SUPERUSER
- DROP
- ALTER
- CREATE

This follows the principle of least privilege by ensuring the application only receives the permissions required to perform its intended functions. Restricting unnecessary privileges reduces the potential impact of compromised credentials and limits accidental changes to the database structure.

Step 4 – Dependency Graph Analysis

Project dependencies were analyzed using the pydeps package and visualized as a dependency graph.

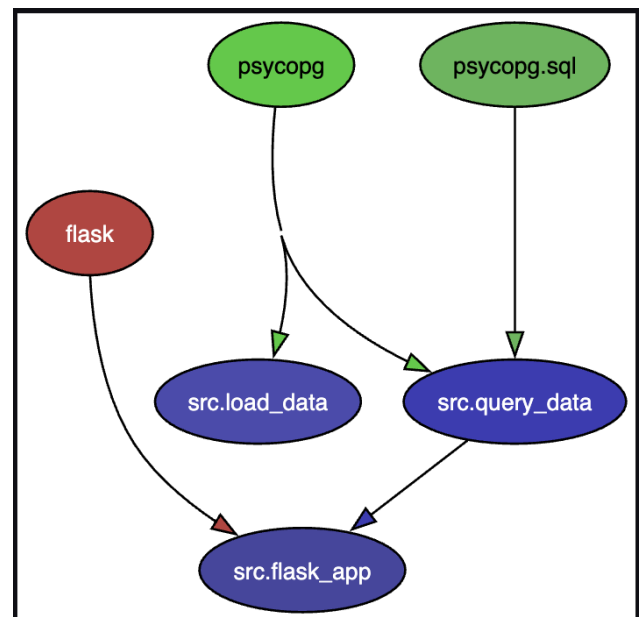
The dependency graph was generated using:

```
pydeps src --noshow -T svg -o dependency.svg
```

The resulting file, dependency.svg, provides a visual representation of relationships between the project's modules and imported packages.

Dependency Graph Analysis

The dependency graph provides insight into how the components of the Grad Cafe Analytics application interact with one another. The Flask application serves as the primary entry point and relies on both the data loading and analytics modules. The load_data.py module is responsible for database population and ETL-related activities, while query_data.py performs analytics queries and reporting functions. Visualizing these dependencies makes it easier to identify tightly coupled modules and potential maintenance concerns. The graph also helps validate that the project structure remains organized and modular. During future development, dependency analysis can be used to identify unnecessary dependencies and reduce complexity. Maintaining a clear dependency structure improves maintainability, testing, and long-term software quality.



Step 5 – Reproducible Environment and Packaging

A reproducible development environment was created to ensure that other developers can reliably install and execute the project.

Installing with pip:

A virtual environment can be created and configured using:

```
python -m venv .venv  
source .venv/bin/activate  
pip install -r requirements.txt
```

Installing with uv:

The same environment can also be created using uv:

```
uv venv  
source .venv/bin/activate  
uv pip install -r requirements.txt
```

Packaging with setup.py:

A setup.py file was created to support packaging and installation of the project.

Benefits:

- Standardized project installation
- Dependency management support
- Easier deployment to new environments
- Better reproducibility
- Compatibility with Python packaging tools

The setup file allows the project to be installed using standard Python tooling and provides a foundation for future distribution.

Step 6 – Snyk Dependency Scan

The Snyk CLI was installed and configured to perform dependency vulnerability scanning.

The following command was executed: snyk test

Results indicated:

Tested 45 dependencies for known issues, no vulnerable paths found.

No known dependency vulnerabilities were identified during the scan.

A screenshot of the scan results is included as:

snyk-analysis.png

```
(.venv) kishore.narayanan@Kishores-MacBook-Air module_5 % snyk test

Testing /Users/kishore.narayanan/Documents/Modern_software_concepts_assignments/jhu_software_concepts/module_5...

Organization:    kishn1234-n9fZcGhKDNTnK7ERNFwh8N
Package manager: pip
Target file:     requirements.txt
Project name:    module_5
Open source:     no
Project path:    /Users/kishore.narayanan/Documents/Modern_software_concepts_assignments/jhu_software_concepts/module_5
Licenses:        enabled

✓ Tested 45 dependencies for known issues, no vulnerable paths found.

Next steps:
- Run `snyk monitor` to be notified about new related vulnerabilities.
- Run `snyk test` as part of your CI/test.
```

Performing vulnerability scans helps identify security risks in both direct and transitive dependencies before software is released into production environments.

Step 7 – CI Enforcement

A GitHub Actions workflow was created to automate quality assurance and security validation.

Workflow file: `.github/workflows/ci.yml`

The workflow automatically performs the following tasks:

1. Installs project dependencies
2. Executes pylint with a required score of 10.00/10
3. Generates and validates the dependency graph
4. Runs Snyk dependency scanning
5. Executes the full Pytest test suite

The workflow is triggered automatically on push and pull request events.

Automating these checks ensures that quality, security, and testing requirements are consistently enforced throughout the development lifecycle. Successful execution of the workflow was verified through GitHub Actions and all pipeline stages completed successfully.

The screenshot shows a GitHub Actions workflow run for the repository 'kishn1234 / jhu_software_concepts'. The workflow is named 'quality-security-tests' and is triggered by a push to the 'main' branch. The run status is 'Success' and it took '1m 4s' to complete. The workflow file is located at '.github/workflows/quality-security-tests.yml'. The run details show a single job 'quality-security-tests' that passed. There is a warning annotation stating: 'Node.js 28 is deprecated. The following actions target Node.js 28 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-python@v5. For more information see: https://github.blog/changelog/2025-09-19-deprecation-of-node-28-on-github-actions-runners/'.

Extra Credit – Snky Code (SAST)

An attempt was made to execute the optional Snky Code Static Application Security Testing (SAST) scan using snky code test

The scan could not be completed because Snky Code was not enabled for the organization associated with the account used for this assignment.

Snky returned:

SNYK-CODE-0005

Snky Code is not supported for your current organization.

Status: 403 Forbidden

```
(.venv) kishore.narayanan@Kishores-MacBook-Air module_5 % snky code test

ERROR Snky Code is not enabled (SNYK-CODE-0005)
This error occurs when Snky Code is not enabled for the current Organization.
Activate Snky Code and try again..

Snky Code is not supported for your current organization: `kishn1234-
n9fZcGhKDNTnK7ERNFwh8N`.

Status: 403 Forbidden
Docs: https://docs.snky.io/scan-with-snky/error-catalog#snky-code-0005
ID: urn:snky:interaction:ac509aa4-bb1c-48bc-9192-c597f4a23170
```

Conclusion

Module 5 focused on strengthening software assurance practices within the Grad Cafe Analytics application. Security improvements included SQL injection protection, query validation, environment-based configuration, dependency analysis, vulnerability scanning, and continuous integration automation.

The final solution achieved:

- 10.00/10 pylint score
- 100% test coverage
- SQL injection defenses
- Dedicated least-privilege database account
- Dependency graph generation
- Successful Snyk vulnerability scan
- Automated GitHub Actions validation

These improvements increase the security, maintainability, reliability, and overall quality of the application while establishing a foundation for future development and deployment.